

Monolithic vs. Agentic vs. Swarm: A Controlled Comparison of Three LLM Execution Architectures

Draft Results Paper

Eric Medlock

University of North Carolina at Charlotte
ITCS 6881 Independent Study
Charlotte, North Carolina, USA
emedlock@charlotte.edu

Dr. Jinzhen Wang

University of North Carolina at Charlotte
Faculty Advisor
Charlotte, North Carolina, USA

Abstract

Large language models (LLMs) can be wired together in many ways to solve a task. The simplest is to ask the model once and take its answer. More elaborate designs let one model check another’s work, or let several copies vote. These designs cost more—in time, in tokens, and in hardware—so a natural question is whether the added machinery actually buys more correct answers, and if so, under what conditions. This paper reports a controlled comparison of three such designs—*monolithic*, *agentic*, and *swarm*—run under identical model, prompt, and decoding settings so that the *architecture* is the only thing that changes. Across a 15-task suite drawn from three standard benchmarks (GSM8K, HumanEval, and HotpotQA), the agentic design reached 100% accuracy and was the best trade-off between accuracy and cost; the monolithic design was cheapest; and the swarm design cost the most while matching, but not beating, the monolithic design. We argue that this last result is an artifact of the tasks being too easy for voting to help, and we describe a harder “frontier” task tier, now under construction, designed to reveal where each architecture genuinely wins.

Keywords

large language models, agentic systems, multi-agent swarm, benchmarking, Pareto analysis, reproducibility

1 Introduction

A large language model (LLM) is a program that, given some text, predicts plausible text to follow it. In practice this single capability—continue the text—can be arranged into very different problem-solving *systems*. One can call the model a single time and use whatever it produces. One can instead have the model produce an answer and then call it again to critique and repair that answer, looping until it is satisfied. Or one can run several independent copies of the model on the same question and take the majority answer, on the theory that independent mistakes cancel out. These arrangements—call them *architectures*—differ enormously in how much time, how many generated tokens, and how much hardware they consume, and it is not obvious that the expensive ones are worth it.

This paper studies that question directly. We fix everything except the architecture: the same underlying model, the same decoding settings (temperature, context window,

token budget), the same tasks, and the same hardware. We then compare three architectures on four axes that matter in practice—accuracy, latency (wall-clock time), token consumption, and hardware resource use—and ask not just *which* architecture is best, but *under what conditions* each one wins. Our guiding hypothesis is that no single architecture dominates universally: the right choice depends on how hard the task is.

The remainder of the paper explains the three architectures (Section 2), describes the benchmark tasks and why they were chosen (Section 3), summarizes how runs were executed and scored (Section 4), presents results (Section 5), interprets them (Section 6), and lays out the remaining work needed to complete the study (Section 8).

2 The Three Architectures

All three architectures use the same underlying LLM; they differ only in how many times the model is called and how those calls are wired together.

Monolithic. The baseline. The task is handed to the model in a single prompt, and its single response is taken as the answer. There is no checking, no retry, and no second opinion. This is the fastest and cheapest arrangement, and it is the one most people mean by “using an LLM.” It serves as the yardstick against which the more elaborate designs are measured.

Agentic. A two-role loop. One instance of the model, the *Executor*, produces a candidate answer. A second instance, the *Verifier*, is prompted specifically to check that answer for errors and either accept it or send it back for revision. If the Verifier is not satisfied, the Executor tries again with the feedback. The loop is capped (at two passes here) so it cannot run forever. The intuition is that catching and fixing mistakes should raise accuracy at the cost of extra model calls and time.

Swarm. A parallel vote. Three independent copies of the model, given the same task, each produce an answer on their own with no communication between them. A simple aggregation step then takes the majority answer. There is no central controller and no ranking of the peers; the design bets that if the individual copies make *different* mistakes, the majority will still land on the correct answer. This is the

most token-hungry design, since it multiplies the work by the number of peers.

Two of the three designs (agentic and swarm) are implemented as small graphs of model calls; the monolithic design is a single call. Crucially, every architecture is accessed through the same software interface and the same underlying model, so any measured difference is attributable to the architecture rather than to incidental implementation choices.

3 Benchmark Tasks

To exercise different kinds of reasoning, the task suite draws five items from each of three widely used benchmarks—15 tasks in total. Each benchmark targets a distinct skill and comes with a definite correct answer, which lets every run be graded automatically.

GSM8K (5 items) — grade-school math word problems. GSM8K (“Grade School Math 8K”) is a well-known collection of arithmetic word problems that require several steps of reasoning—not just recalling a fact, but working through a short chain of calculations to reach a single numeric answer (for example, totaling costs, splitting quantities, or computing a rate). These items test multi-step numerical reasoning. They are graded by *exact numeric match*: the model’s final number either equals the reference answer or it does not.

HumanEval (5 items) — Python programming. HumanEval is a standard code-generation benchmark in which each item gives a function signature and a docstring describing what the function should do, and the model must write the body. What makes HumanEval convenient is that every item ships with hidden *unit tests*: a solution is correct only if the generated code actually runs and passes those tests. This is the strictest form of grading in the suite—the answer is executed, not merely compared—and it tests the model’s ability to produce working, logically correct programs.

HotpotQA (5 items) — multi-hop question answering. HotpotQA is a question-answering benchmark whose questions cannot be answered from a single fact. Each question requires combining two or more pieces of information—“hops”—to reach the answer (for example, identifying an entity from one clue and then looking up a property of that entity). These items test reading comprehension and the chaining of facts. They are graded by *normalized string match*, which compares the model’s answer to the reference after standardizing capitalization, spacing, and punctuation so that trivial formatting differences do not count as wrong.

Together the three benchmarks span numeric reasoning, program synthesis, and multi-hop factual reasoning, giving a small but deliberately varied picture of where each architecture helps or hurts. The suite is intentionally compact so that every task can be run many times under every architecture within a reasonable time and hardware budget.

4 Method in Brief

Every one of the 15 tasks was run under every one of the 3 architectures, and each of those combinations was

repeated $N = 5$ times to average out run-to-run variation— $15 \times 3 \times 5 = 225$ runs in total. All runs used a single open-weights model, **deepseek-r1-distill-qwen-14b** (a 14-billion-parameter model, 4-bit quantized), served locally through LM Studio on an Apple M5 Max with 48 GB of unified memory, using the machine’s Metal GPU backend. Decoding was held fixed across all runs (temperature 0, an 8,192-token context, and a 6,144-token generation budget) so that the model behaved as deterministically as possible.

Each run recorded not only the answer but also its latency, the number of tokens consumed, and a snapshot of hardware telemetry (memory and GPU use). Answers were scored two ways. First, an *automatic grader* appropriate to the benchmark (exact-numeric, unit-test, or normalized-string, as described above) produced an objective correct/incorrect verdict. Second, as a cross-check, a separate and deliberately different model—Llama-3.2-3b, chosen from a different model family than the backend—acted as an “LLM-as-judge,” independently rating each answer. Using a different-family judge guards against a model simply agreeing with its own style. The judge’s verdicts agreed with the automatic grader 93–100% of the time, which gives confidence that the automatic scores are trustworthy.

5 Results

Table 1 summarizes the headline numbers, averaged over all tasks and repetitions. Accuracy is the fraction of runs graded correct; latency is mean wall-clock seconds per run (with standard deviation); tokens is the mean number of tokens generated per run.

Table 1: Results across the 15-task baseline suite (225 runs). Bold marks the best value in each column.

Architecture	Accuracy	Latency (s)	Tokens
Monolithic	93%	15.2 ± 17.5	832
Agentic	100%	32.5 ± 32.0	1961
Swarm	93%	32.9 ± 33.1	2649

Three observations stand out. First, the **agentic** architecture was the only one to get every task right (100%), at roughly double the time and tokens of the monolithic baseline. Second, the **monolithic** architecture was by far the cheapest—fastest and fewest tokens—while still getting 93% correct. Third, the **swarm** architecture was the most expensive of all (most tokens, comparable latency to agentic) yet matched, and did not beat, the monolithic baseline’s 93% accuracy.

Put in trade-off terms: agentic and monolithic each sit on the “Pareto frontier”—the set of options for which you cannot get more accuracy without paying more, or pay less without losing accuracy. Agentic is the choice when accuracy matters most; monolithic is the choice when cost matters most. The swarm, on this suite, is dominated: it costs more than monolithic for the same accuracy.

6 Discussion

The swarm result deserves scrutiny, because on its face it suggests majority voting is simply not worth it. We believe the more accurate reading is that *these particular tasks are too easy for voting to help*. Majority voting only rescues a task when the individual copies disagree—some right, some wrong—so that the correct answer can win the vote. On tasks the model almost always gets right on the first try, all copies already agree, and the vote merely re-confirms an answer the cheap monolithic design would have produced anyway. The extra copies add cost without adding information. In other words, the swarm’s poor showing is a property of the *task selection*, not a verdict on the architecture.

This directly motivates the central hypothesis of the study: architecture superiority is *task-dependent*. On easy tasks, the cheapest design that gets the answer wins, and elaboration is wasted. On genuinely hard tasks—where the model is right only some of the time—checking (agentic) and voting (swarm) have room to change the outcome. The baseline suite establishes the easy end of that spectrum; it cannot, by construction, reveal the hard end.

7 Project Status

At the time of writing, the study is at the end of its first phase. Concretely:

- **v1 data collection: complete.** All 225 baseline runs (15 tasks $\times$ 3 architectures $\times$ 5 repetitions) have been executed, hardware-stamped, and independently scored by the LLM-as-judge. The dataset is frozen and will not change.
- **v1 analysis: complete.** Accuracy, latency, token, and judge-agreement figures are finalized and reported above; the accuracy-cost Pareto picture is settled.
- **Software harness: stable.** The shared backend interface, the three architectures, the automatic graders, the telemetry capture, the judge, and the analysis pipeline are all implemented and in use, backed by an offline test suite.
- **Frontier task tier: designed, not yet run.** A candidate set of 11 harder items exists but is uncalibrated; this is the active work item and the critical path to the study’s central result.
- **Cloud/GPU environment and framework migration: designed, not started.** Provisioning scripts exist; the multi-agent-framework and ensemble-swarm extensions are specified but unimplemented.

In short: the easy end of the difficulty spectrum is fully measured, and the infrastructure needed to measure the hard end is built. The one thing standing between the current results and the study’s main hypothesis is calibrating and running the frontier tier.

8 Next Steps

Frontier task tier (in progress). The immediate priority is a harder task tier deliberately chosen to sit in the regime where architecture matters. A candidate set of 11 harder items (covering math, code, and multi-hop reasoning) has been assembled. Each candidate will be calibrated by running the monolithic design on it several times and measuring how often it succeeds; only items where the single-pass success rate falls in a middle band—hard enough to fail sometimes, easy enough to succeed sometimes—will be kept, since those are exactly the tasks where checking and voting can flip the result. Once this tier is frozen, all three architectures will be run and scored on it, which is expected to produce the task-by-architecture picture the baseline suite cannot.

Cloud / GPU comparison. The current results come entirely from Apple Metal hardware. Provisioning scripts for an NVIDIA GPU cloud instance already exist but have not been run; executing them would add a second hardware environment and let us compare the architectures’ resource behavior across very different accelerators.

Planned extensions. Two larger extensions are designed but not yet executed. The first re-implements the agentic and swarm designs on established multi-agent frameworks (CrewAI and OpenAI Swarm respectively) rather than the current hand-built versions; because the framework itself affects overhead, those results would be reported as their own baseline rather than compared directly to the numbers here. The second adds an *ensemble* swarm—voting across not just multiple copies of one model but multiple prompt variants and multiple distinct models—to test whether diversity among voters, rather than mere repetition, is what makes voting pay off.

9 Conclusion

Holding everything but the architecture fixed, we found that on a suite of moderate-difficulty tasks the agentic “check-and-repair” design bought perfect accuracy at a real but modest cost premium, the plain monolithic design was the efficient choice, and the parallel-voting swarm design paid more for no accuracy gain—an outcome we attribute to the tasks being too easy for voting to matter. The work confirms the qualitative shape of our hypothesis and points squarely at the missing piece: a harder task tier, now under construction, that should reveal whether and where the more elaborate architectures earn their cost.

AI Usage Statement

AI tools were used during this project in a supporting role, with all research design, decisions, and interpretation remaining the authors’ responsibility. Specifically, LLM-based coding assistants were used to help scaffold and refactor the experimental harness, write tests, and format results; and an AI assistant was used to help draft and copy-edit the prose of this report. The empirical results themselves were produced

by the experimental system described in Section 4, not generated by an AI assistant. Notably, an LLM also appears *inside* the methodology—as the model under test in all three architectures, and as the independent LLM-as-judge used to

cross-check scoring; those uses are experimental instruments and are documented as such. All AI-assisted text and code were reviewed and verified by the authors before inclusion.